

Objectif du test

Créer une application Django nommée cinema permettant de :

- Gérer des films, auteurs, spectateurs
- Fournir une interface d'administration complète
- Offrir une API REST complète pour interagir avec le système
- Intégrer des données externes via l'API de **The Movie Database (TMDb)**

Pré-requis techniques

- Python ≥ 3.12
- Django ≥ 4.2
- Django REST Framework $\geq 3.15.2$
- PostgreSQL
- Docker + docker-compose

Modèles

Inspirez vous des informations de l'API TMDb pour améliorer les champs de vos modèles. Ci-dessous un exemple.

1. Auteur

- (nom, email, date de naissance)
- Basé sur AbstractUser (auth personnalisé)

2. Film

- (titre, description, date de sortie, evaluation, auteur associé, statut)
- L'évaluation est une liste de choix possible (à vous de les définir)
- Le statut est une liste de choix possible (à vous de les définir)

3. Spectateur

- Basé sur AbstractUser (auth personnalisé)
- Champs personnalisés possibles (bio, avatar, etc.)
- Un spectateur peut noter un film et un auteur

Fonctionnalités attendues

1. Interface d'administration (Django admin)

- CRUD complet sur tous les modèles
- **Filtres avancés pour les Films :**
 - Par date de création
 - Par évaluation
 - Par statut
- **Filtres pour les Auteurs :**
 - Ceux ayant au moins un film
 - Dans la fiche d'un auteur : liste des films associés
 - Dans la fiche d'un film : liste des auteurs et des notations
- **Filtres pour les listes des Spectateurs :**
 - Avoir des inlines pour les favoris dans l'admin vue détail des spectateurs

2. API REST (Django REST Framework)

Endpoints requis :

- Pouvoir lister les auteurs
- Pouvoir récupérer les informations pour un seul auteur
- Pouvoir modifier les informations d'un auteur
- Pouvoir supprimer un auteur uniquement s'il n'a pas de livre écrit
- Pouvoir lister les films
- Pouvoir lister les films avec un statut bien précis (exemple : publié)
- Pouvoir récupérer les informations pour un seul film
- Pouvoir modifier les informations d'un film
- Pouvoir archiver un film
- Pouvoir s'inscrire en tant qu'un spectateur
- Pouvoir s'authentifier en tant qu'un spectateur
- Pouvoir se déconnecter de sa session spectateur
- Pouvoir noter un film en tant qu'un spectateur
- Pouvoir noter un auteur en tant qu'un spectateur
- Pouvoir ajouter un film à la liste de films favoris

- Pouvoir retirer un film à la liste de films favoris
- Pouvoir lister ses films favoris en tant que spectateur
- Authentification JWT pour les appels API

3. Intégration API TMDb

- Intégration récupération des films via API The Movie Database
- Avoir une commande django qui récupère les informations de films, auteurs afin de les enregistrer en base
- Avoir un filtre dans la route API des films et auteurs pour récupérer uniquement ceux qui sont créés depuis l'admin ou depuis import via TMDb

4. Exigences techniques

- Utiliser **Django** et **Django REST Framework**
- Utiliser **PostgreSQL** comme base de données
- L'API doit renvoyer des réponses au format **JSON**
- Utiliser les Serializers imbriqués afin de remonter les informations des objets liés (***pas d'utilisation de SerializerMethodField***)
- Un utilisateur anonyme (non authentifié) peut avoir accès en lecture seule de tous les films et auteurs
- Utiliser JWT comme système d'authentification pour les appels API le nécessitant
- Utiliser docker pour conteuneuriser l'application

4. Livrables attendus

- Code source dans un repo **Git** public (**gitlab** ou **github**)
- Un fichier **README.md** contenant :
 - Instructions de lancement du projet
 - Description des endpoints disponibles

Bonne chance à vous et beaucoup de courage.